

Multilingual Low-resource Grapheme-to-Phoneme Conversion

Aosimanjiang Aihaiti

School of Computation, Information and Technology
Technical University of Munich (TUM)
a.aihaiti@tum.de

Yanbo Luo and Yi Wu

School of Computer and Control Engineering
University of Chinese Academy of Sciences (Visiting Student at TUM)
{luoyanbo22, wuyi22}@mailsucas.ac.cn

Abstract

Grapheme-to-phoneme (G2P) conversion serves as a critical component in numerous language processing technologies, including text-to-speech systems. However, to date, such tasks typically require substantial data resources for robust performance, rendering G2P conversion particularly challenging for low-resource languages. Building upon the resources and solutions provided by the SIGMORPHON Shared Task (Gorman et al., 2020), this study aims to explore methods for enhancing G2P performance under multilingual low-resource conditions. We investigated model architectures suited for this particular setting and proposed approaches that leverage cross-linguistic relationships to achieve this objective.

1 Introduction

Grapheme-to-phoneme (G2P) conversion is a computational process that maps sequences of graphemes (smallest functional unit of a writing system) to sequences of phonemes (the smallest sound units, represented in the International Phonetic Alphabet). As a core component of text-to-speech (TTS) and automatic speech recognition (ASR) systems, G2P plays a crucial role in modern speech technologies.

However, despite the growing societal demand for high-quality text-to-speech conversion, existing G2P solutions struggle to meet current needs. This challenge arises from two primary factors. First, creating a comprehensive grapheme-phoneme mapping is a high-cost endeavor. This process requires extensive manual annotation by linguistic experts, which is both time-consuming and expensive, making it difficult to build and maintain large-scale, accurate databases. This problem is compounded by the resource scarcity for many languages, where publicly available linguistic resources are so limited that the existing data is often insufficient to support

even basic G2P applications, making robust modeling difficult.

To address these challenges, this study aims to explore methods for improving phoneme prediction accuracy in a low-resource setting. Specifically, we simulated the low-resource setting by constraining dataset size. Similar to the SIGMORPHON Shared Task, our research focused on multilingual approach. Through systematic experimentation, we evaluated the performance of diverse models in performing G2P conversion on our curated multilingual dataset. We also employed script conversion techniques (e.g. character decomposition and transliteration) to investigate on the impact of variant grapheme forms on G2P performance. Moreover, to enhance target-language G2P accuracy, we curated combined multilingual datasets by aggregating linguistically related languages. This approach leveraged language family relationships to improve cross-lingual transfer learning, particularly for low-resource scenarios. By developing models capable of generalizing from limited data, we seek to contribute potential solutions that enhance G2P performance and broaden its applicability.

2 Related Work

2.1 Key Contributions from SIGMORPHON

As a top-performing submission in the 2020 SIGMORPHON Shared Task (Gorman et al., 2020), Makarov and Clematide (2020)’s system was subsequently adopted as the official baseline for the 2021 and 2022 tasks. The architecture, which we also adopt, utilizes a *neural transducer* framework, bidirectional Long Short-Term Memory (LSTM) (Hochreiter and Schmidhuber, 1997) encoder with attention mechanism and LSTM decoder generating explicit edit operations. This approach demonstrated impressive performance in grapheme-to-phoneme conversion through its dynamic operation sequencing.

Girrbach (2023)’s system (Tü-G2P) introduced innovations in sequence labeling for word-level morpheme segmentation and a hybrid architecture of Gram-CTC (Liu et al., 2017). This system achieved significant accuracy improvements, particularly in low-resource languages.

2.2 Multilingual attempts based on Language Relation

Several researchers have attempted to leverage language phylogenetic relationships to construct approximate G2P models and achieve better low-resource G2P performance through joint training (Li et al., 2022). Their general approach involves building language family trees based on the Glotlog database (Nordhoff and Hammarström, 2012), calculating the shortest path distances between languages, then selecting the top-k phylogenetically the closest training languages for the target language, and finally integrating the outputs of multiple models through an improved ROVER system (Fiscus, 1997).

While drawing inspiration from this approach, our work extends beyond linguistic genealogy alone by incorporating additional dimensions such as orthographic similarity and historical-cultural connections in our experiments.

3 Data

3.1 Data Sources

Our data sources align with those used in the SIGMORPHON Shared Task. The grapheme-phoneme pairs were extracted from the dataset in the WikiPron repository (Lee et al., 2020), a large-scale G2P database with standardized formatting. This resource derives linguistic data from Wiktionary, a comprehensive online multilingual dictionary. The pronunciation resources in Wiktionary originate from three potential sources, varying by language: Existing authoritative dictionaries, computer-generated pronunciations based on established phonological rules, manual entries and adjustments by volunteer contributors.

3.2 Data Processing

The data from the WikiPron database is stored in TSV files and each line adheres to a ‘{Grapheme}\t{Phoneme}’ format, where phonemes are space-delimited to prevent segmentation ambiguity. The 49 specific languages utilized in our experiments are listed in the Appendix A.

To simulate a low-resource scenario, we limit the data to exactly 1,000 grapheme-phoneme pairs per language. These pairs are randomly shuffled to eliminate dictionary-order bias before being split into training (800 samples), validation (100 samples), and test (100 samples) sets.

For multilingual experiments, we combine the monolingual datasets and introduce an explicit language identifier for each entry. Specifically, we prepend a three-letter ISO 639-3 *language tag*, enclosed in special separators (e.g., <|ENG|> for English), to each grapheme string. The final line format is ‘<|TAG|>{Grapheme}\t{Phoneme}’ (Example in Appendix B). This tagging approach serves a purpose that to provide explicit language identity signals to the model, which enables language-specific processing while maintaining a unified multilingual training framework.

3.3 Language-Specific Processing Tools

To evaluate G2P performance across diverse writing systems, we incorporated several publicly available language processing tools in our experiments:

- **Python-Jamo**¹: Decomposes Hangul characters into *jamo* (the fundamental components of the Korean writing system)
- **Epitran** (Mortensen et al., 2018) and **Uroman** (Hermjakob et al., 2018): Rule-based tools for transliterating multiple scripts into Latin characters

In corresponding experiments, we replaced original script representations with their transformed versions to create modified datasets (Example in Appendix C). This approach allowed us to standardize orthographic representations for cross-lingual comparison and isolate the impact of writing system characteristics on G2P performance.

4 Models

We used three different models in our experiments. The metric for the model performance is word error rate (WER), defined as $WER = (\#Incorrect\ Predictions) / (\#Total\ Words) \times 100\%$.

4.1 Transformer

A standard Transformer-based sequence-to-sequence model as implemented in fairseq library (Ott et al., 2019), with both encoder and

¹<https://github.com/JDongian/python-jamo>

decoder consisting of 4 layers and 4 self-attention heads per layer. We tie decoder input/output embeddings, use pre-layer normalization and ReLU activations, and apply a uniform dropout on both attention and activation. Training is carried out with label-smoothed cross-entropy ($\varepsilon = 0.1$) and the Adam optimizer under an inverse-square-root decay schedule (1000 warmup steps, peak $LR = 1 \times 10^{-3}$, gradient clipping to norm 1.0). Inference uses beam search (beam size = 5). This model is one of the baseline models from the 2020 shared task (Gorman et al., 2020).

Hyperparameter Tuning We optimized the model architecture by tuning the size of the encoder and decoder on the development set. We established a 2x2 hyperparameter grid by defining two model sizes: ‘small’ and ‘large’, with dimensions specified in Table 1. Due to computational constraints, this tuning process was performed on six representative languages. The full results are visualized in Appendix D.

Although the performance of models with a large decoder improved consistently when paired with a large encoder, the lowest WER for five of the six languages was ultimately achieved using a small decoder. Of these top-performing models, four also benefited from a large encoder. Based on these findings, we selected the *large encoder/small decoder* configuration for training all final models.

Parameter	Small Config.	Large Config.
Embedding Dim	128	256
FFN Hidden Dim	512	1024

Table 1: Parameter dimensions for the ‘small’ and ‘large’ model configurations.

4.2 Transducer

A neural transducer system built upon an imitation learning paradigm (Makarov and Clematide, 2018). For training, alignments are first computed using 10 iterations of expectation maximization. The system’s policy is then trained for up to 60 epochs with patience of 12, employing the Adadelta optimizer. During the prediction phase, it utilizes a beam search with a size of 4. Final outputs are determined by a 10-component ensemble, where predictions are aggregated through a majority vote. A notable technical detail is that all internal processing is handled using the decomposed Unicode

normalization form (NFD), while the final predictions are converted back to the standard composed form (NFC). This transducer served as the official baseline for the SIGMORPHON 2021 and 2022 shared tasks (Ashby et al., 2021; McCarthy et al., 2023).

4.3 Attentive LSTM

A BiRNN (Schuster and Paliwal, 1997) model with a single bidirectional LSTM encoder layer and a single LSTM decoder layer, connected via a standard attention mechanism, implemented using the yoyodyne library². Both encoder and decoder use 128-dimensional input embeddings and have a hidden size of 512. To mitigate overfitting, dropout with a rate of 0.2 is applied, and training employs early stopping with patience of 10 epochs.

5 Method

5.1 Monolingual Baselines and Model Performance Comparison

To establish performance baselines, we trained individual models for each of the 49 languages in our dataset. This approach allows for a direct comparison of the performance of the three architectures under consistent training conditions using average WER of all languages, and also provides a reference point for our subsequent multilingual experiments.

5.2 Multilingual Experiments

5.2.1 Effect of Language Tag Script on Model Performance

Creating multilingual datasets with non-Latin-script languages led us to investigate whether converting the tag from Latin script to the language’s native script would improve model performance. To test this, we trained models on a dataset of Cyrillic-script Slavic languages (Russian, Ukrainian, and Bulgarian). We then compared model performance across four configurations: (1) no tag, (2) a lower-case Latin-script tag, (3) an uppercase Latin-script tag, and (4) an uppercase native Cyrillic-script tag (Example in Appendix F).

5.2.2 Comparison with Monolingual Baselines

To evaluate the impact of multilingual training, we created several datasets, each consisting of three related languages (e.g., a Germanic dataset with German, Swedish, and Dutch). We trained models on these multilingual datasets and then compared

²<https://github.com/CUNY-CL/yoyodyne>

the WER for each language to that of its monolingual counterpart. This analysis aimed to determine whether training with typologically similar languages improves model performance compared to single-language training.

5.2.3 Cross-Family Language Transfer

To investigate whether multilingual training can benefit languages from different families, we performed two sets of experiments.

Our primary experiment focused on a curated dataset of Maltese, Italian, and transliterated Arabic. The rationale for this selection is the unique historical relationship between these languages: Maltese has Semitic roots (related to Arabic) but has been extensively influenced by Italian (a Romance language). By comparing the performance of each language in this multilingual setting to its respective monolingual baseline, we could assess the extent of cross-family transfer.

In a second, unplanned observation, we noted positive transfer between Uyghur (a Turkic language) and Persian (an Indo-Iranian language). Our Persian data includes both Classical and modern Iranian variants. We analyzed these two variants separately to determine if the known historical influence of Classical Persian on Uyghur would correlate with a stronger transfer effect.

5.3 Transliteration Experiment

To explore how transliteration affects G2P model performance, we performed the following steps:

Establish a Monolingual Baseline We transliterated several non-Latin-script languages using the tools described in Section 3.3. We then compared the WER of models trained on this transliterated data against models trained on the languages’ native scripts. This comparison isolates the impact of transliteration.

Evaluate Multilingual Training We then investigated whether multilingual training could enhance performance for these transliterated languages. To do this, we created new datasets by combining transliterated data with data from related Latin-script languages and compared their WER against the monolingual baselines. We experimented with three different multilingual combinations (see Appendix G for the full list of languages). We first created *Slavic Latin* dataset, which combined transliterated Cyrillic-script and native Latin-script Slavic languages. Next, we built an *Indo-European Latin*

dataset, a broader dataset of transliterated and native Latin-script languages from the Indo-European family. Finally, we compiled *All-Arabic (Transliterated)*, a dataset of various Arabic-script languages all transliterated into Latin script. This last dataset was designed to address a key challenge in G2P for abjads like Arabic, where the native script omits short vowels and inconsistently marks consonant length. We wanted to see if making these features explicit through transliteration, combined with cross-lingual transfer, could mitigate this issue.

6 Results and Discussion

6.1 Monolingual Experiments: Model Performance Comparison

These experiments demonstrate that the neural transducer consistently outperforms other models, achieving an average WER of 28.98%. This is a marked improvement over the transformer (35.23%) and the attentive LSTM (32.06%), confirming the transducer’s robustness in low-resource monolingual G2P settings.

However, model performance varies considerably across languages. In Korean, for instance, the neural transducer significantly outperforms both baselines on Hangul data, yet all three models perform comparably on decomposed Jamo data (Table 2). This contrast suggests that the transducer’s strength lies in its ability to model complex, non-linear orthographic rules, such as Hangul’s syllabic block structure, which are simplified in the Jamo representation. Conversely, languages with highly regular orthographies, such as Georgian and Uyghur, yield near-perfect results across all models, with error rates around 1.25%. Detailed results for all languages are provided in Appendix A.

	Transducer	Transformer	LSTM
Hangul	23	89	100
Jamo	23	28	28

Table 2: Model Performance on Hangul and Jamo Data

6.2 Multilingual

6.2.1 Effect of Language Tag Script

As shown in Table 3, explicit language tags are essential for effective multilingual modeling. Without a tag, all models yield a high WER of 45%. With the introduction of tags, the WER for the transformer and attentive LSTM models drops substantially to

Tag Script	Transducer	Transformer	LSTM
No Tag	45	45	45
UP Cyrillic	40	28	28
UC Latin	31	28	27
LC Latin	32	28	27

Table 3: Model Performance on different Language Tag

approximately 28%. For these two architectures, performance remained stable regardless of the tag’s script, which is likely due to the subword tokenizer that processes the language tag into a distinct token (or set of tokens), making the script of the original graphemes irrelevant.

In contrast, the neural transducer model demonstrated a significant sensitivity to the tag’s script. Using a native Cyrillic-script tag caused its WER to increase by 8% compared to using a Latin-script tag. We hypothesize that this degradation occurs because the transducer confuses the Cyrillic tag with the actual text, as they share the same character set. A Latin tag provides a clearer, out-of-domain signal, preventing this confusion and allowing the model to perform optimally. This suggests that for transducer-based models, using a different script for language tags is a crucial implementation detail.

6.2.2 Comparison to monolingual Baselines

Average WER Reduction	
Transducer	-5.11
Transformer	2.44
LSTM	1.33

Table 4: Average WER reduction of multilingual models compared to monolingual baselines

Our findings (Table 4) indicate that both the Transformer and LSTM architectures benefited from multilingual training, achieving an average WER reduction of 2.44% and 1.33%, respectively. This improvement is likely due to a combination of factors. On one hand, it suggests that these models successfully leveraged linguistic similarities from related languages, resulting in positive cross-lingual transfer. However, it is also important to acknowledge that the multilingual training regimen inherently increases the total volume of training data, which a data-hungry architecture like the Transformer benefits from.

In stark contrast, the Transducer model exhibited

a significant performance degradation, with a WER increase of 5.11%. This strongly suggests that the performance loss is not due to data scarcity but rather to negative transfer, where the model struggles to disentangle the subtle phonetic and grammatical variations between the related languages. This interference appears to have led to a confused internal representation that outweighed any potential benefit from the increased data volume.

6.2.3 Cross-Family Language Transfer

Maltese, Italian and transliterated Arabic Multilingual training yielded a clear and significant benefit for both Arabic and Maltese. The most dramatic improvement was for transliterated Arabic, where the Attentive LSTM model achieved a remarkable 18% absolute reduction in WER, and the Transducer model a 12% reduction. Maltese also benefited greatly, with the Transformer and Attentive LSTM models improving its performance by 7% and 6% absolute WER, respectively. In contrast, Italian performance did not improve; it either remained the same or degraded (Table 5).

The significant gains for both Arabic and Maltese, contrasted with the lack of improvement for Italian, highlight the impact of their shared Semitic roots.

	Transducer	Transformer	LSTM
Italian	-9	0	-2
Maltese	-6	7	6
Arabic	12	-2	18

Table 5: WER Improvement of each Language

Persian and Uyghur Our experiment co-training Uyghur and Persian revealed a strong, positive transfer to Persian, an effect that was disproportionately larger for Classical Persian than for modern Iranian Persian, as depicted in Figure 6. Specifically, the Transformer model achieved a remarkable 16% WER reduction for Classical Persian, compared to just 2% for Iranian Persian. The Attentive LSTM model showed a similar, albeit less pronounced, trend with a 6% WER reduction for Classical Persian and 2% for Iranian Persian.

This disparity in performance strongly correlates with the historical linguistic context. Modern Uyghur inherited a substantial lexicon of loanwords from Classical Persian, which served as the lingua franca for culture and administration in Central Asia in that era. In contrast, its contact with

Persian	Transducer	Transformer	LSTM
Classical	-1	16	6
Iranian	-3	2	2

Table 6: WER Improvement of Persian languages when co-trained with Uyghur language

modern Iranian Persian has been negligible (Aisaiti, 2020). The results suggest our models, particularly the Transformer, were able to leverage these shared historical lexical features. Since Uyghur’s highly regular orthography results in a near-zero WER in monolingual settings, the benefits of co-training were primarily observed on the Persian side.

Interestingly, this underlying pattern held even with the Neural Transducer model, which showed overall negative transfer. The performance degradation was less severe for Classical Persian (-1% WER) than for Iranian Persian (-3% WER), further reinforcing that the historical connection consistently mediates the degree of transfer.

6.3 Script and Transliteration

6.3.1 Monolingual Models

Our monolingual experiments reveal that transliteration, when applied in isolation, is largely detrimental to G2P model performance. Across the 10 non-Latin-script languages tested, performance improvements were rare. For the neural transducer and transformer architectures, only 2 of the 10 languages benefited from transliteration. The attentive LSTM model performed slightly better, showing improvement in 4 of the 10 languages (See appendix G.1 for the full result table). This clear trend suggests that transliteration introduces an abstraction layer or potential information loss that most models struggle with in a monolingual context. Therefore, we conclude that using transliteration as a standalone pre-processing step is generally not advisable.

6.3.2 Multilingual Models

In stark contrast to the monolingual results, multilingual training significantly alters the impact of transliteration. For the transformer and attentive LSTM models, combining transliterated languages with related Latin-script languages yielded consistent gains. The average WER improved across all three multilingual datasets (Slavic Latin, Indo-European Latin, and All-Arabic), with relative improvements ranging from 1.86% to 15%.

Conversely, the neural transducer model failed to benefit from this multilingual, transliterated context, showing negative average WER changes in all setups. This result reinforces our earlier finding from Section 6.2.2, confirming that the neural transducer architecture is less adept at leveraging cross-lingual transfer, even when orthographic differences are harmonized via transliteration. (Full result in Appendix G.2)

6.3.3 The Case of Arabic

The Arabic-script languages provide a compelling case study for the interaction between transliteration and multilingual training. As established by our monolingual results, simply transliterating Arabic to make vowels explicit did not improve—and in fact, worsened—model performance. This suggests that the challenge of predicting omitted vowels in the abjad script cannot be solved by script conversion alone.

However, a different pattern emerged in the multilingual All-Arabic (Transliterated) setting. Despite the poor monolingual baseline, the multilingual models trained on the combined transliterated Arabic languages achieved WERs comparable to their counterparts trained on native Arabic scripts. This key finding indicates that while making phonological features explicit via transliteration is insufficient on its own, its combination with cross-lingual data transfer effectively compensates for the initial performance degradation. The model can thus leverage both the explicit phonological cues from transliteration and the shared patterns across related languages to overcome the inherent ambiguity of the native abjad script.

Conclusion

Our study yields three key findings: (1) Under monolingual conditions, the Neural Transducer model achieves optimal performance for low-resource G2P tasks; (2) For Transformer and LSTM architectures, multilingual training incorporating related languages demonstrates significant performance enhancement. (3) Transliteration techniques show negative effects in monolingual G2P scenarios but exhibit some improvements in multilingual contexts.

Acknowledgments

We wish to express our gratitude to our supervisor, Dr. Shu Okabe, for their invaluable guidance

throughout our project and for their insightful feedback on the poster and this report.

References

- Sulaiman Aisaiti. 2020. [Loanwords in uyghur in a historical and socio-cultural perspective](#). *Uluslararası Uygar Araştırmaları Dergisi*, (15):31–69. Section 2.2.
- Lucas F.E. Ashby, Travis M. Bartley, Simon Clematide, Luca Del Signore, Cameron Gibson, Kyle Gorman, Yeonju Lee-Sikka, Peter Makarov, Aidan Malanoski, Sean Miller, Omar Ortiz, Reuben Raff, Arundhati Sengupta, Bora Seo, Yulia Spektor, and Winnie Yan. 2021. [Results of the second SIGMORPHON shared task on multilingual grapheme-to-phoneme conversion](#). In *Proceedings of the 18th SIGMORPHON Workshop on Computational Research in Phonetics, Phonology, and Morphology*, pages 115–125, Online. Association for Computational Linguistics.
- Jonathan G Fiscus. 1997. A post-processing system to yield reduced word error rates: Recognizer output voting error reduction (rover). *1997 IEEE Workshop on Automatic Speech Recognition and Understanding Proceedings*, pages 347–354.
- Leander Gierbach. 2023. [SIGMORPHON 2022 shared task on grapheme-to-phoneme conversion submission description: Sequence labelling for G2P](#). In *Proceedings of the 20th SIGMORPHON workshop on Computational Research in Phonetics, Phonology, and Morphology*, pages 239–244, Toronto, Canada. Association for Computational Linguistics.
- Kyle Gorman, Lucas F.E. Ashby, Aaron Goyzueta, Arya McCarthy, Shijie Wu, and Daniel You. 2020. [The SIGMORPHON 2020 shared task on multilingual grapheme-to-phoneme conversion](#). In *Proceedings of the 17th SIGMORPHON Workshop on Computational Research in Phonetics, Phonology, and Morphology*, pages 40–50, Online. Association for Computational Linguistics.
- Ulf Hermjakob, Jonathan May, and Kevin Knight. 2018. [Out-of-the-box universal Romanization tool uroman](#). In *Proceedings of ACL 2018, System Demonstrations*, pages 13–18, Melbourne, Australia. Association for Computational Linguistics.
- Sepp Hochreiter and Jürgen Schmidhuber. 1997. [Long short-term memory](#). *Neural Computation*, 9(8):1735–1780.
- Jackson L. Lee, Lucas F.E. Ashby, M. Elizabeth Garza, Yeonju Lee-Sikka, Sean Miller, Alan Wong, Arya D. McCarthy, and Kyle Gorman. 2020. [Massively multilingual pronunciation modeling with WikiPron](#). In *Proceedings of the Twelfth Language Resources and Evaluation Conference*, pages 4223–4228, Marseille, France. European Language Resources Association.
- Xinjian Li, Florian Metze, David Mortensen, Shinji Watanabe, and Alan Black. 2022. [Zero-shot learning for grapheme to phoneme conversion with language ensemble](#). In *Findings of the Association for Computational Linguistics: ACL 2022*, pages 2106–2115, Dublin, Ireland. Association for Computational Linguistics.
- Hairong Liu, Zhenyao Zhu, Xiangang Li, and Sanjeev Satheesh. 2017. [Gram-CTC: Automatic unit selection and target decomposition for sequence labelling](#). In *Proceedings of the 34th International Conference on Machine Learning*, volume 70 of *Proceedings of Machine Learning Research*, pages 2188–2197. PMLR.
- Peter Makarov and Simon Clematide. 2018. [Imitation learning for neural morphological string transduction](#). In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, pages 2877–2882, Brussels, Belgium. Association for Computational Linguistics.
- Peter Makarov and Simon Clematide. 2020. [CLUZH at SIGMORPHON 2020 shared task on multilingual grapheme-to-phoneme conversion](#). In *Proceedings of the 17th SIGMORPHON Workshop on Computational Research in Phonetics, Phonology, and Morphology*, pages 171–176, Online. Association for Computational Linguistics.
- Arya D. McCarthy, Jackson L. Lee, Alexandra DeLucia, Travis Bartley, Milind Agarwal, Lucas F.E. Ashby, Luca Del Signore, Cameron Gibson, Reuben Raff, and Winston Wu. 2023. [The SIGMORPHON 2022 shared task on cross-lingual and low-resource grapheme-to-phoneme conversion](#). In *Proceedings of the 20th SIGMORPHON workshop on Computational Research in Phonetics, Phonology, and Morphology*, pages 230–238, Toronto, Canada. Association for Computational Linguistics.
- David R. Mortensen, Siddharth Dalmia, and Patrick Littell. 2018. [Epitran: Precision G2P for many languages](#). In *Proceedings of the Eleventh International Conference on Language Resources and Evaluation (LREC 2018)*, Miyazaki, Japan. European Language Resources Association (ELRA).
- Sebastian Nordhoff and Harald Hammarström. 2012. [Glottolog/langdoc:increasing the visibility of grey literature for low-density languages](#). In *Proceedings of the Eighth International Conference on Language Resources and Evaluation (LREC’12)*, pages 3289–3294, Istanbul, Turkey. European Language Resources Association (ELRA).
- Myle Ott, Sergey Edunov, Alexei Baevski, Angela Fan, Sam Gross, Nathan Ng, David Grangier, and Michael Auli. 2019. fairseq: A fast, extensible toolkit for sequence modeling. In *Proceedings of NAACL-HLT 2019: Demonstrations*.
- M. Schuster and K.K. Paliwal. 1997. [Bidirectional recurrent neural networks](#). *IEEE Transactions on Signal Processing*, 45(11):2673–2681.

A Language List and Result

Table 7 shows all the language that we have in our dataset and used for monolingual model training, as well as the result for each language and model.

B Dataset Example

Table 8 shows an example how the multilingual dataset was created from the monolingual data.

C Example for Script Processing Tools

C.1 Hangul to Jamo

Table 9 shows an example how a word from the Korean (Hangul) dataset is decomposed to Jamo using the Python-Jamo library and the word looks like in both datasets.

C.2 Arabic Transliteration

Table 10 shows an example how non-Latin script words are transliterated using the Epritrans and Uroman transliteration tools.

D Transformer Hyperparameter Tuning Result

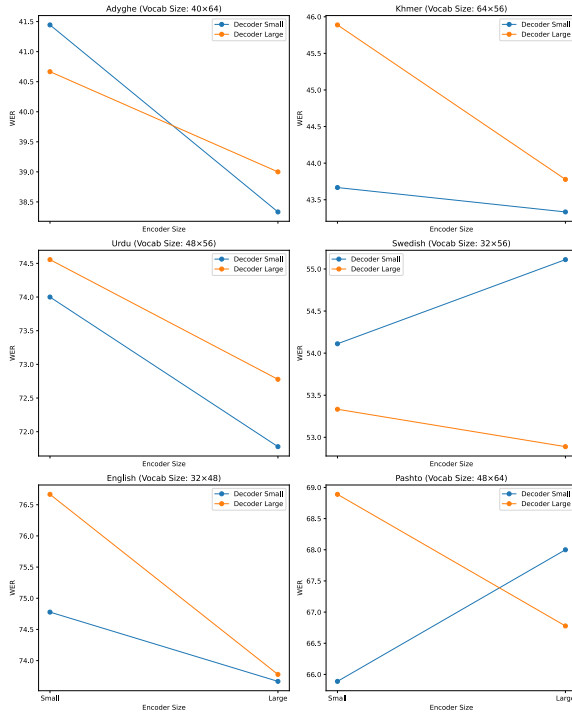


Figure 1: Encoder-Decoder Interaction by Language (Graphemes x Phonemes)

Language	Transducer	Transformer	LSTM
Adyghe	20	39	30
Arabic	53	43	45
Assamese	7	15	12
Belarusian	2	6	2
Bengali	68	78	67
Bulgarian	32	30	27
Burmese	29	41	35
Cebuano	20	26	20
Central Khmer	31	43	35
Classical Persian	57	58	51
Dutch	24	29	23
Eastern Armenian	15	16	20
Eastern Lawa	8	24	12
English	63	70	66
French	23	29	27
Georgian	0	4	1
German	46	56	45
Greek	20	26	27
Hindi	11	24	13
Hungarian	7	11	8
Icelandic	12	24	16
Indonesian	64	52	73
Iranian Persian	65	63	65
Irish	43	46	39
Italian	15	21	16
Japanese	10	20	12
Korean	23	89	100
Latvian	51	51	50
Lithuanian	33	35	31
Macedonian	5	5	5
Maltese	17	24	22
Norwegian Nynorsk	66	69	69
Pashto	67	68	66
Polish	4	8	7
Romanian	9	17	17
Russian	23	21	25
Serbo-Croatian	64	69	69
Shan	5	10	4
Slovenian	56	52	50
Spanish	4	10	3
Swedish	59	68	61
Tagalog	13	17	11
Thai	39	49	42
Ukrainian	19	26	18
Urdu	72	66	77
Uyghur	0	3	2
Vietnamese	5	20	4
Welsh	12	20	19
avg WER	28.98	35.23	32.06

Table 7: Monolingual Result & Language List

dunk	d ʌ ŋ k
foolish	f u l ɪ ʃ
cervello	tʃ e r v ɛ l l o
importante	i m p o r t a n t e
отсчётах	a tʃ ɛ : o t a x
разживалась	r a z i v a ʎ a sʲ

< ENG >dunk	d ʌ ŋ k
< ENG >foolish	f u l ɪ ʃ
< ITA >cervello	tʃ e r v ɛ l l o
< ITA >importante	i m p o r t a n t e
< RUS >отсчётах	a tʃ ɛ : o t a x
< RUS >разживалась	r a z i v a ʎ a sʲ

Table 8: Dataset Example: Monolingual to Multilingual

Hangul	Jamo
덩치	ㄷ ㅅ ㅅ ㅅ ㅅ
덩치	t ʌ ŋ tɕʰ i
ㄷ ㅅ ㅅ ㅅ ㅅ	t ʌ ŋ tɕʰ i

Table 9: Hangul to Jamo Decomposition

Arabic	Uroman	Epitrans	IPA
تشيلي	tshyly	tʃi:li:	tʃi:li:jj
توفيق	twfyq	tu:fi:q	tawfi:q

Table 10: Transliteration Example of two Arabic words

E Additional Experiment: Impact of Data Scaling on Performance

E.1 Monolingual

Method While our work focuses on low-resource scenarios, we also investigated how performance scales with data volume. To do this, we trained the three architectures on English datasets of nine different sizes, ranging from 1k to 40k utterances (1k, 1.5k, 2k, 3k, 4k, 5k, 10k, 20k, 40k). We then compared the WER of the models to understand the relationship between data availability and performance for each architecture.

Result This experiment shows that all models improve as the dataset grows. Notably, the transformer narrows the performance gap and even surpasses the transducer when the dataset is sufficiently large (Figure 2), suggesting that attention-based architec-

tures can better leverage abundant resources. This trend indicates a trade-off: while the transducer dominates in low-resource conditions, the transformer has greater capacity to scale with data.

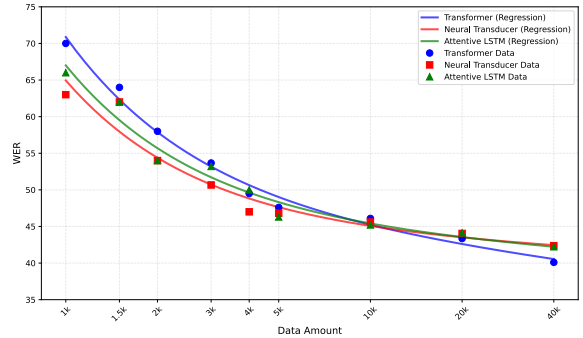


Figure 2: Models WER Performance Across Varying English Dataset Sizes

E.2 Multilingual

Method To study the impact of data scaling, we expanded two of our multilingual datasets. Each dataset, which originally contained three related languages, was extended with a fourth related language (e.g., from the same family or script). We then trained models on these new four-language datasets. To evaluate the effect, we compared the average WER improvement of three initial languages for the models.

Result Table 11 shows that adding a fourth language improved the average WER reduction for all models. The Attentive LSTM and Transformer models saw modest gains, reinforcing that they benefit from multilingual data. The most significant improvement was for the Neural Transducer, whose WER reduction increased from 0.67% to 1.67%. Given our earlier finding (Section 6.2.2) that the Transducer suffers from negative transfer, we conclude that this gain is primarily due to the increased overall data volume rather than improved cross-lingual transfer. This suggests that while more data is broadly beneficial, the underlying reasons for improvement differ across architectures.

	Transducer	Transformer	LSTM
3 Languages	0.67	-1.00	4.33
4 Languages	1.67	-0.67	4.67

Table 11: Average WER reduction for models trained on three vs. four related languages

F Example for different Language Tag

No Tag	обилие
UC Cyrillic	< PYC >обилие
UC Latin	< RUS >обилие
LC Latin	< rus >обилие

Table 12: A Russian word prepended with different script Language Tag

Table 12 show how a word looks like in the multilingual dataset when prepended with uppercase-Cyrillic, uppercase-Latin and lowercase-Latin script language tag.

G Language list for the Transliteration Experiment

G.1 Monolingual Result

G.1.1 Slavic Latin

See Table 13.

	Transducer	Transformer	LSTM
Bulgarian	-2	-5	2
Russian	3	-1	5
Ukrainian	-1	2	2
Macedonian	-1	0	0

Table 13: WER Reduction for Languages in the *Slavic Latin* Dataset comparing to their native script Baselines - **Monolingual**

G.1.2 Indo-European Latin

See Table 14.

	Transducer	Transformer	LSTM
Eastern Armenian	-25	-27	-19
Greek	-10	-9	-8

Table 14: WER Reduction for Languages in the *Indo-European Latin* Dataset comparing to their native script Baselines - **Monolingual**

G.1.3 All Arabic

See Table 15 and 16.

G.2 Multilingual Result

G.2.1 Slavic Latin

See Table 17.

	Transducer	Transformer	LSTM
Arabic	-17	-24	-25
Classical Persian	9	9	1
Pashto	-7	-8	-6
Urdu	0	-1	-1
Uyghur	-28	-35	-33

Table 15: WER Reduction for Languages in the *All Arabic* Dataset comparing to their native script Baselines - **Monolingual** and using **Uroman** for transliteration

	Transducer	Transformer	LSTM
Arabic	-10	3	-11
Classical Persian	-1	10	-5
Pashto	/	/	/
Urdu	-1	-1	2
Uyghur	-4	-29	-2

Table 16: WER Reduction for Languages in the *All Arabic* Dataset comparing to their native script Baselines - **Monolingual** and using **Epitrans** for transliteration

	Transducer	Transformer	LSTM
Bulgarian*	3	8	4
Russian*	1	-2	5
Ukrainian*	-1	6	-1
Serbo-Croatian	-12	6	8
Polish	-9	1	-1
Slovenian	2	-5	1
Macedonian*	2	-2	1
avg	-2.00	2.29	2.43

Table 17: WER Reduction for Languages in the *Slavic Latin* Dataset comparing their monolingual Baselines

G.2.2 Indo-European Latin

See Table 18.

G.2.3 All Arabic

See Table 19 and 20.

H Contribution

In Table 21, we state the contribution of each member with “+”. In case the contribution was not even for a task, the more “+” symbol indicates more contribution from the member.

	Transducer	Transformer	LSTM
Classical Persian*	-10	-1	4
Russian*	-19	-7	-5
Italian	-6	2	1
Eastern Armenian*	-5	3	-3
German	-1	11	-6
Greek*	-2	2	2
Irish	-2	3	-2
avg	-6.43	1.86	-1.29

Table 18: WER Reduction for Languages in the *Indo-European Latin* Dataset comparing their monolingual Baselines

	Transducer	Transformer	LSTM
Arabic*	-5	23	20
Classical Persian*	-13	11	3
Urdu*	-4	2	9
Uyghur*	-12	0	2
avg	-8.5	9	8.5

Table 19: WER Reduction for Languages in the *All Arabic* Dataset comparing their monolingual Baselines - **Uroman**

	Transducer	Transformer	LSTM
Arabic*	5	2	20
Classical Persian*	-8	5	12
Urdu*	-4	4	13
Uyghur*	-6	27	0
avg	-3.3	9.5	15

Table 20: WER Reduction for Languages in the *All Arabic* Dataset comparing their monolingual Baselines - **Epitran**

	Aosimanjiang	Yanbo	Yi
Implementation			
Neural Transducer	+		+
Attentive LSTM		+	
Transformer	+		
Data Processing			+
Literature Research	+	+	+
Poster			
Introduction	+	+	+
Data Processing	+		+
Monolingual Results		+	
Multilingual Results	+		+
Design & Typeset	+		+
Poster Session	++	+	+
Report			
Abstract			+
Introduction	+		+
Related Work			+
Data			+
Models	+		
Method	+		
Results	++	+	
Conclusion			+
Appendix	++	+	+

Table 21: Contributions of group members for each task